

Late Breaking Results: On the One-Key Premise of Logic Locking

Yinghua Hu Hari Cherupalli Mike Borza Deepak Sherlekar

Synopsys, Inc., Sunnyvale, CA, USA
{yinghuah, harich, mborza, dsherlek}@synopsys.com

ABSTRACT

The evaluation of logic locking methods has long been predicated on an implicit assumption that only the correct key can unveil the true functionality of a protected circuit. Consequently, a locking technique is deemed secure if it resists a good array of attacks aimed at finding this correct key. This paper challenges this one-key premise by introducing a more efficient attack methodology, focused not on identifying that one correct key, but on finding multiple, potentially incorrect keys that can collectively produce correct functionality from the protected circuit. The tasks of finding these keys can be parallelized, which is well suited for multi-core computing environments. Empirical results show our attack achieves a runtime reduction of up to 99.6% compared to the conventional attack that tries to find a single correct key.

1 INTRODUCTION

The past two decades have witnessed rapid development of logic locking, a promising hardware intellectual property (IP) protection method to thwart various threats, e.g., IP piracy and hardware Trojan insertion, performed by rogue agents on the semiconductor supply chain [1, 2]. Logic locking techniques are designed to secure a design's netlist by incorporating extra programmable logic components that are regulated through an additional set of input ports called key ports. If a wrong key value is applied to these key ports, the altered netlist will produce an incorrect function. This mechanism enables designers to safely send the locked netlist to a foundry without the risk of IP theft in the supply chain. Evaluating the effectiveness of logic locking has led to many efficient attacks, e.g., [3, 4], aimed at discovering the correct unlocking key. Traditionally, a locking method is deemed "secure" if it withstands a wide array of these attacks. However, this conventional wisdom hinges on a critical assumption, i.e., the one-key premise, which posits that obtaining the correct key is essential for accessing the original design's functionality. Surprisingly, this fundamental assumption has received scant attention in literature, raising questions about its validity and the potential implications for the security of logic locking.

This paper scrutinizes the conventional "one-key" premise and introduces a novel attack methodology based on the Boolean satisfiability (SAT) attack methodology [5]. Unlike traditional approaches that focus on identifying the single correct key, our method seeks to identify multiple incorrect keys that, when used together, can effectively unlock the protected design as if the correct key were applied. Furthermore, this proposed attack can be formulated as a set of independent sub-tasks, which significantly shortens the attack time in a multi-core environment—a capability readily exploitable by resource-rich adversaries in the supply chain. Our empirical evidence shows that the proposed attack can achieve up to 99.6% runtime reduction, demonstrating that, under the "multi-key" assumption, many existing logic locking techniques can be much less secure than they appear.

2 PRELIMINARIES & BACKGROUND

We denote the original circuit as C , with the sets of input and output ports labeled as I and O , respectively. Consequently, the function of

C , denoted by f , is defined as $f: \mathbb{B}^{|I|} \rightarrow \mathbb{B}^{|O|}$. For the locked circuit, represented as C_l , its function f_l incorporates an additional set of key ports, K , introduced during the locking process, and is given by $f_l: \mathbb{B}^{|I|} \times \mathbb{B}^{|K|} \rightarrow \mathbb{B}^{|O|}$. The correct unlocking key, k^* , satisfies the condition $f_l(i, k^*) = f(i)$, $\forall i \in \mathbb{B}^{|I|}$, distinguishing it from incorrect keys that produce erroneous outputs for at least one input pattern.

Satisfiability Attack. The SAT attack [5] is among the most effective attacks against logic locking. The attack assumes two primary resources: (1) the reverse-engineered locked design netlist C_l from the GDSII file used by the foundry, and (2) the original design's golden input-output pairs, obtainable through querying a commercially available chip. This attack initiates by constructing a miter circuit from duplicate instances of C_l , which is then analyzed by a SAT solver. The solver's objective is to identify a specific input pattern, known as a Distinguishing Input Pattern (DIP), that exposes output discrepancies between the two instances of C_l under different keys. Each identified DIP, along with its corresponding output obtained from the "oracle", i.e., the working chip, helps eliminate incorrect keys by refining the miter circuit. The process iterates until no further DIPs can be found, indicating the successful elimination of all incorrect keys from the search space. A key in the remaining search space will be returned as the correct key. Since its creation, the SAT attack has become a standard method to evaluate new logic locking techniques. The count of DIPs needed to uncover the correct key, symbolized as $\#DIP$, serves as a preliminary gauge of a locking method's security. Additionally, certain locking techniques, like LUT-based insertion [6], complicate the SAT attack by increasing the miter instance's complexity after each iteration, thus leading to longer SAT solving time over successive iterations. Consequently, the duration of the attack is often employed as a more precise measure of a locking technique's resilience.

Countermeasures against the SAT Attack. Numerous strategies have been developed to counter the SAT attack, broadly falling into two categories: (1) augmenting the required number of DIPs, and (2) enhancing the complexity of the miter to identify a DIP. *SARLock* [7] is an example of the first category, adding a "point function" component to the original design. While *SARLock* corrupts only a small portion of the design function, $\#DIP$ exponentially grows with the key size. Conversely, to markedly elevate the complexity of the miter, various computationally demanding logic elements, such as Look-up Tables (LUTs) [6] and eFPGAs [8], are embedded into the original design netlist. While attacking these enhanced schemes may not necessitate a large $\#DIP$, the resolution of each DIP demands a considerable amount of time. In this paper, we benchmark our proposed attack against *SARLock* and LUT-based insertion.

3 ATTACK METHODOLOGY

As a foundation of our improved attack, we explore the feasibility of utilizing multiple incorrect keys to access the full design functionality. This approach is predicated on the idea that the locked design's function, f_l , can be decomposed into several terms, where each term is contingent upon distinct conditions applied to the primary inputs. For instance, one such condition might involve the most significant bit (MSB) of the input ports being set to either a logical high or low state. We represent the corresponding decomposed function of f_l as follows:

$$f_l(i, k) = f_l(i_{[\text{msb}=0]}, k) \vee f_l(i_{[\text{msb}=1]}, k), \quad (1)$$

where each term encapsulates one half of the overall locked function f_l . Similarly to f_l , the two sub-functions defined in equation (1) can be unlocked with the correct key. However, it is notable that each sub-function might also be accessible using different incorrect keys.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions.acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to Association for Computing Machinery.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

<https://doi.org/10.1145/3649329.3663504>

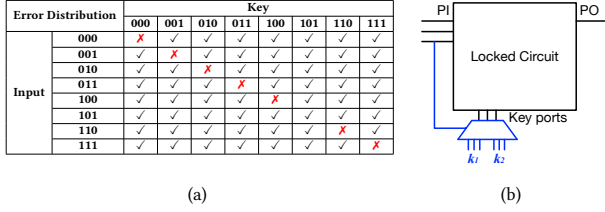


Figure 1: (a) Error distribution on an example circuit ($|I| = |K| = 3$), and (b) the schematic of the modified locked netlist.

Algorithm 1 Proposed Attack Flow

Input: Locked netlist C_L , black-box oracle C , splitting effort N

Output: A list of N keys $\mathcal{K}$

```

1:  $\mathcal{K} = \text{list}()$ 
2: for  $i = 1$  to  $2^N$  do
3:    $b = \text{convert\_to\_binary\_and\_pad}(i, N)$ 
4:    $C_{temp} = \text{generate\_conditional\_netlist}(C_L, b)$ 
5:    $k = \text{SAT\_attack}(C_{temp}, C)$ 
6:    $\mathcal{K}.\text{append}(k)$ 
7: return  $\mathcal{K}$ 

```

For instance, consider the error distribution for a 3-input circuit locked with SARLock, as depicted in Fig. 1(a). Here, the sole correct key is 101. Yet, by examining the sub-function for when the MSB is zero (the top four rows), we identify three incorrect keys (100, 110, and 111) that also successfully unlock this portion of the function. A similar pattern emerges when analyzing the sub-function for the MSB set to one (the bottom four rows). Suppose the attacker already possesses two keys, each capable of unlocking a different half of the design’s function. Fig. 1(b) illustrates a schematic of the adjusted locked netlist. In this netlist, the two keys serve as input to a multiplexer (MUX) module, which operates based on the same criterion used for function splitting—specifically, the MSB’s logic value. This alteration renders the netlist functionally equivalent to the one unlocked with the correct key, underscoring the potential for the collective usage of multiple, potentially incorrect keys to access the full design function.

Algorithm 1 outlines our enhanced SAT attack designed to identify multiple keys, with each key unlocking a specific portion of the locked function. The attacker inputs the locked netlist C_L , an oracle C , and a splitting effort N , thereby dividing the input space into 2^N distinct terms. Each term is contingent on setting N primary input ports to the binary sequence b generated in line 3 and is subsequently synthesized to remove any redundant logic (line 4). The resulting netlist, C_{temp} , undergoes the traditional SAT attack process (line 5). The key reported by the SAT attack is then added to the key list $\mathcal{K}$ (line 6). The algorithm concludes by repeating the aforementioned steps for all the 2^N distinct terms. Although Algorithm 1 is presented as a sequential process, it is worth highlighting that the tasks within each distinct term are autonomous and can be executed independently of one another. By leveraging the capacity for parallel execution, attackers can adjust the splitting effort N in alignment with their computational resources, significantly reducing the total duration of the attack.

4 VALIDATION RESULTS

Our enhanced attack is primarily implemented in PYTHON, with MINISAT utilized for SAT solving to maintain consistency with the traditional SAT attack method. For the synthesis of the modified locked netlists, we employ the Synopsys Design Compiler and the 45-nm Nangate Open Cell Library [9]. To ensure a fair comparison, we use the ISCAS’85 benchmark circuits [10], aligning with the datasets reported in both SARLock [7] and LUT-based insertion studies [6]. All experiments are executed on a server with 16×2.6 -GHz cores, leading to the range of the splitting effort N to be from 1 to 4. The selection of which N input ports to apply the splitting condition is determined through a fan-out cone analysis of the netlist’s input ports, prioritizing those with the most key-controlled gates in their fan-out cones. This approach is based on the premise that input ports influencing wider

Table 1: #DIP Results for SARLock-locked c7552.

	$N = 0$ (baseline)	$N = 1$	$N = 2$	$N = 3$	$N = 4$
$ K = 4$	14	6	3	1	1
$ K = 8$	255	127	63	31	15
$ K = 12$	4095	2047	1023	511	255

Table 2: Runtime (seconds) of Attacking LUT-based Insertion.

Circuit	Baseline [5]	This work			
		Minimum	Mean	Maximum	Maximum/Baseline
c880	42311.8	80.1	127.1	169.9	0.004
c1355	26533.5	84.0	133.7	213.2	0.008
c1908	15005.3	124.6	152.9	198.2	0.013
c2670	91403.3	17763.6	38103.8	57304.0	0.627
c3540	158416.0	123.5	314.0	560.3	0.004
c5315	1774.5	665.8	2358.9	5627.4	3.171
c6288	8652.8	0.172	102.3	213.3	0.025
c7552	48815.3	170.0	316.8	544.1	0.011

key-controlled areas can significantly simplify the netlist’s logic when held constant, thereby further reducing the complexity of miters used in the SAT solving.

Attacking SARLock. Since the number of DIPs (#DIP) required to break SARLock is deterministic, we first apply our enhanced attack on SARLock as a flow checker. Table 1 shows the results of attacking a benchmark circuit c7552, locked with different key sizes. When $N > 0$, we observe the same #DIP for all the parallelized tasks. It is obvious to see that #DIP exponentially decreases as N goes up.

Attacking LUT-based Insertion. Aligned with the locking methodology described in [6], we incorporate a 14-input 2-stage LUT module into the benchmark circuits, effectively equating to a key size of 156. To maximize the advantage of our proposed attack, we use a splitting effort of $N = 4$. Finally, we present in Table 2 the runtime of the traditional SAT attack and our proposed attack. Within our proposed attack, we detail the minimum, maximum, and average of the 16 parallel tasks. Given that our attack’s efficiency is determined by the runtime of the most time-intensive sub-task, we report our method’s advantage using the ratio of the maximum sub-task runtime to that of the baseline attack. Remarkably, except for c5315, all other benchmarks exhibit an average runtime reduction of 90.1%, with the maximum reduction being as high as 99.6%. For six out of the eight benchmarks, the maximum-to-baseline runtime ratio falls below $1/16 = 0.0625$, suggesting significant efficiency improvement even within a single-core computing context. This result primarily stems from dividing the key searching task into smaller segments, which not only result in smaller SAT instances to solve but also increase the likelihood of more keys capable to unlock the sub-functions.

5 CONCLUSION

We challenge the traditional “one-key” premise in logic locking techniques with an innovative attack strategy capable of efficiently identifying multiple, potentially incorrect keys that, collectively, can unlock the protected design. Future works include creating effective defenses to counter the new “multi-key” attack scenario.

REFERENCES

- [1] Y. Hu *et al.*, “Risk-aware cost-effective design methodology for integrated circuit locking,” in *Design, Automation and Test in Europe Conference and Exhibition (DATE)*, pp. 1182–1185, IEEE, 2021.
- [2] H. M. Kamali *et al.*, “Advances in logic locking: Past, present, and prospects,” Cryptology ePrint Archive, Paper 2022/260, 2022. <https://eprint.iacr.org/2022/260>.
- [3] Y. Hu *et al.*, “On the security of sequential logic locking against oracle-guided attacks,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 11, pp. 3628–3641, 2023.
- [4] D. Chen *et al.*, “Unraveling latch locking using machine learning, Boolean analysis, and ILP,” in *24th Int. Symp. Quality Electronic Design (ISQED)*, pp. 1–8, IEEE, 2023.
- [5] P. Subramanyan *et al.*, “Evaluating the security of logic encryption algorithms,” in *IEEE Int. Symp. Hardware Oriented Security and Trust (HOST)*, pp. 137–143, 2015.
- [6] S. D. Chowdhury *et al.*, “Enhancing SAT-attack resiliency and cost-effectiveness of reconfigurable-logic-based circuit obfuscation,” in *Int. Symp. Circuits and Systems (ISCAS)*, pp. 1–5, IEEE, 2021.
- [7] M. Yasin *et al.*, “SARLock: SAT attack resistant logic locking,” in *IEEE Int. Symp. Hardware Oriented Security and Trust (HOST)*, pp. 236–241, 2016.
- [8] P. Mohan *et al.*, “Hardware redaction via designer-directed fine-grained eFPGA insertion,” in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1186–1191, IEEE, 2021.
- [9] Silvaco, “45nm open cell library.” Available at <https://si2.org/open-cell-library/>.
- [10] M. C. Hansen *et al.*, “Unveiling the ISCAS-85 benchmarks: A case study in reverse engineering,” *IEEE Design & Test of Computers*, vol. 16, no. 3, pp. 72–80, 1999.